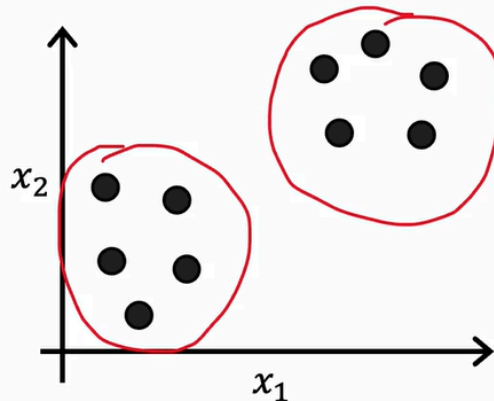


Clustering

What is clustering?

Unsupervised learning



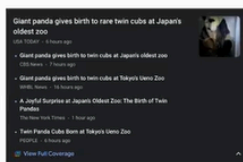
Clustering

Training set: $\{x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)}\}$

In unsupervised learning, the dataset does not have the label.

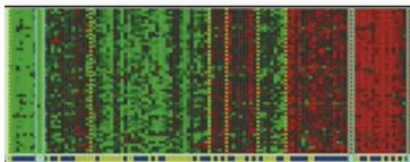
→ The first algorithm we learn is "Clustering"

Applications of clustering

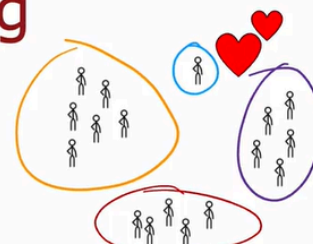


Grouping similar news

- Growing skills
- Develop career
- Stay updated with AI, understand how it affects your field of work



DNA analysis



Market segmentation



Image credit: NASA/JPL-Caltech/E. Churchwell (Univ. of Wisconsin, Madison)

Astronomical data analysis

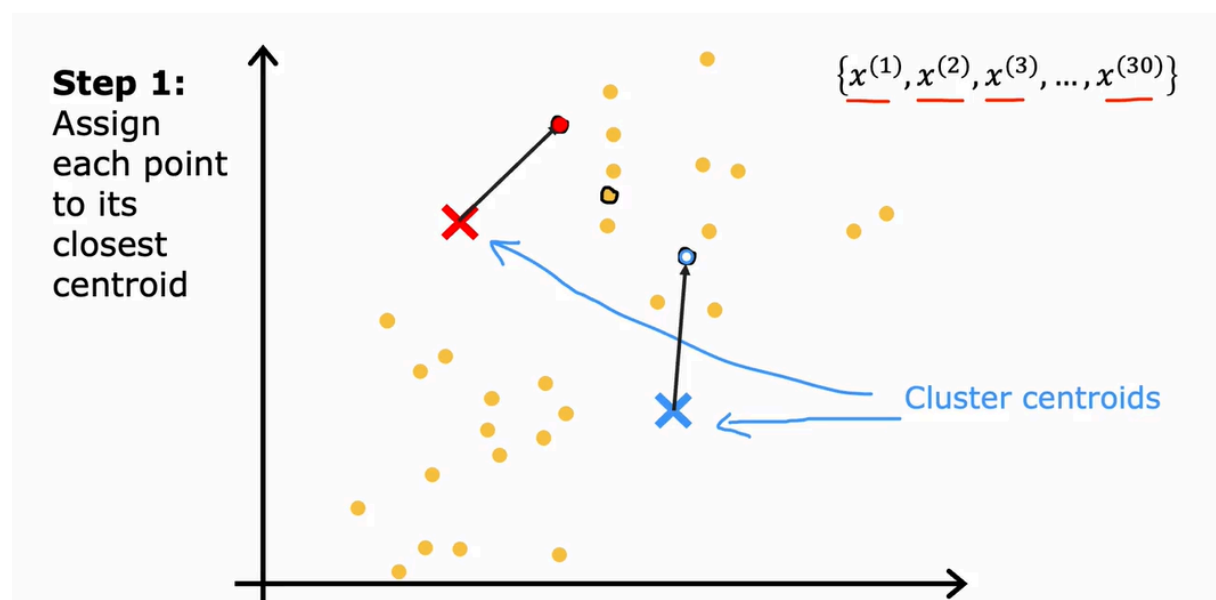
Here are some applications of clustering. In the first week of the first course, you heard me talk about grouping similar news articles together, like the story

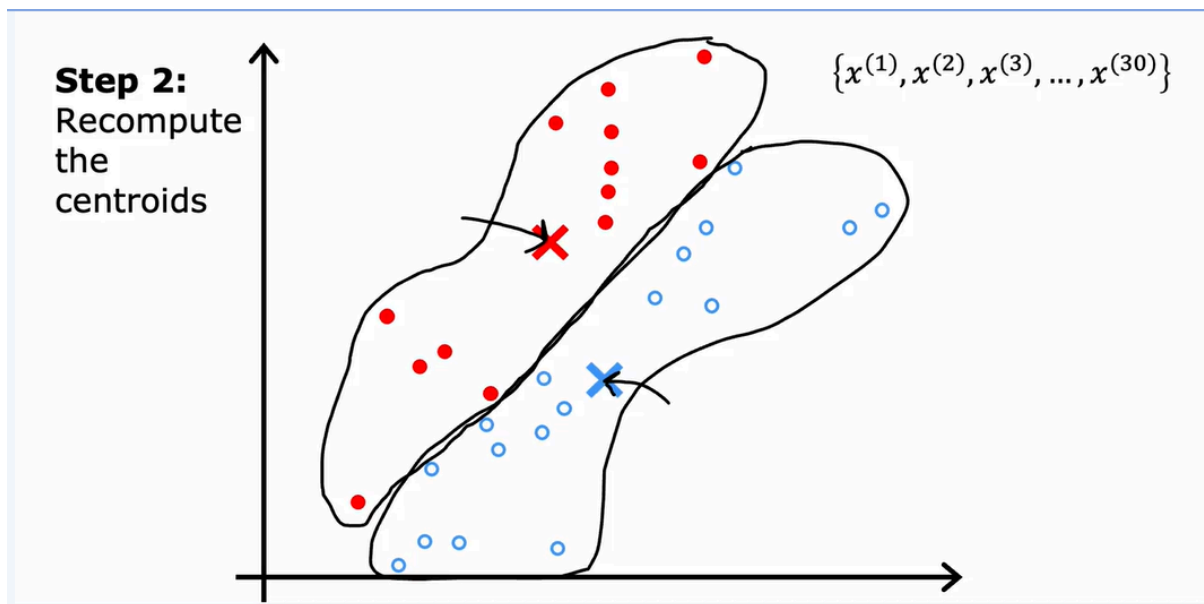
about Pandas or market segmentation, where at deeplearning.ai, we discovered that there are many learners that come here because you may want to grow your skills, or develop your careers, or stay updated with AI and understand how it affects your field of work. We want to help everyone with any of these skills to learn about machine learning, or if you don't fall into one of these clusters, that's totally fine too. I hope deeplearning.ai and Stanford Online's materials will be useful to you as well.

Clustering has also been used to analyze DNA data, where you will look at the genetic expression data from different individuals and try to group them into people that exhibit similar traits. I find astronomy and space exploration fascinating. One application that I thought was very exciting was astronomers using clustering for astronomical data analysis to group bodies in space together for their own analysis of what's going on in space.

One of the applications I found fascinating was astronomers using clustering to group bodies together to figure out which ones form one galaxy or which one form coherent structures in space.

K-means intuition

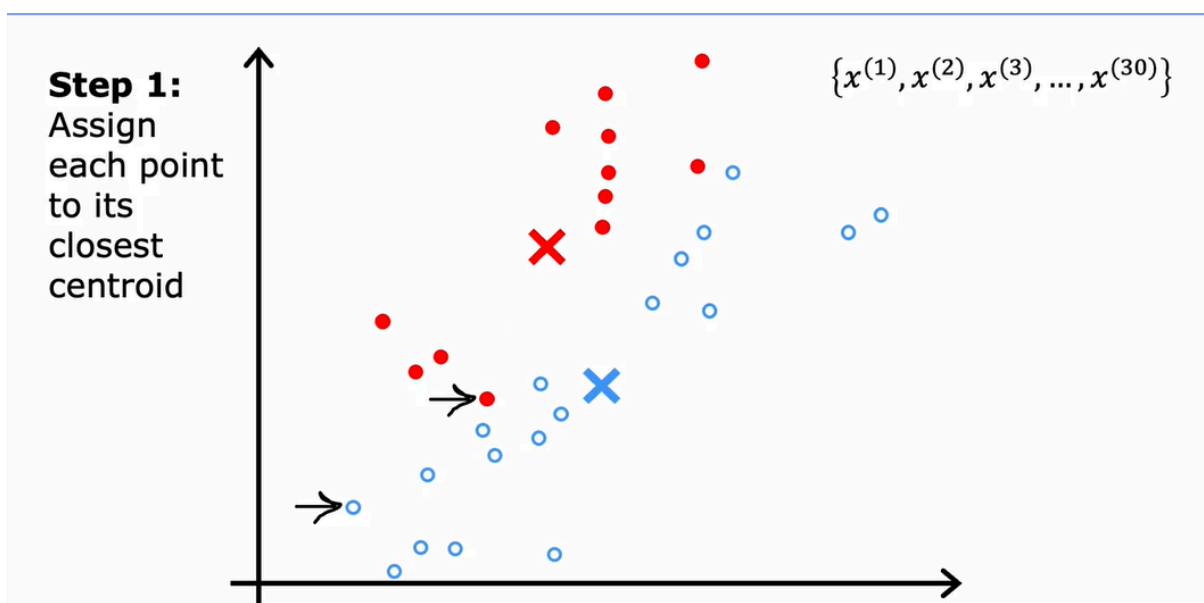


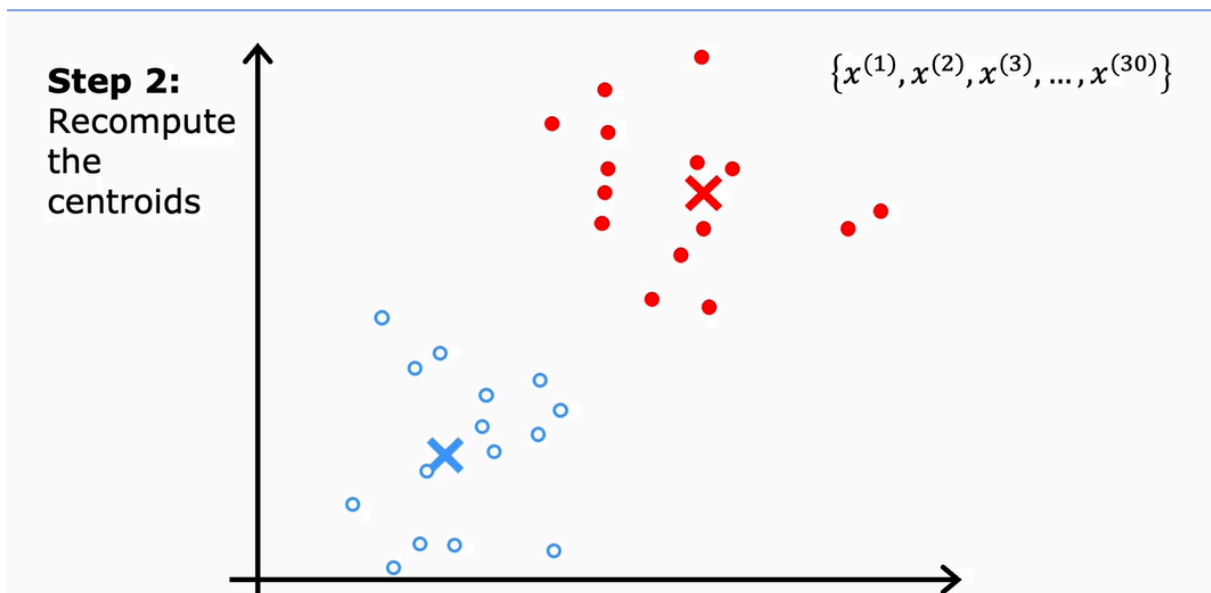
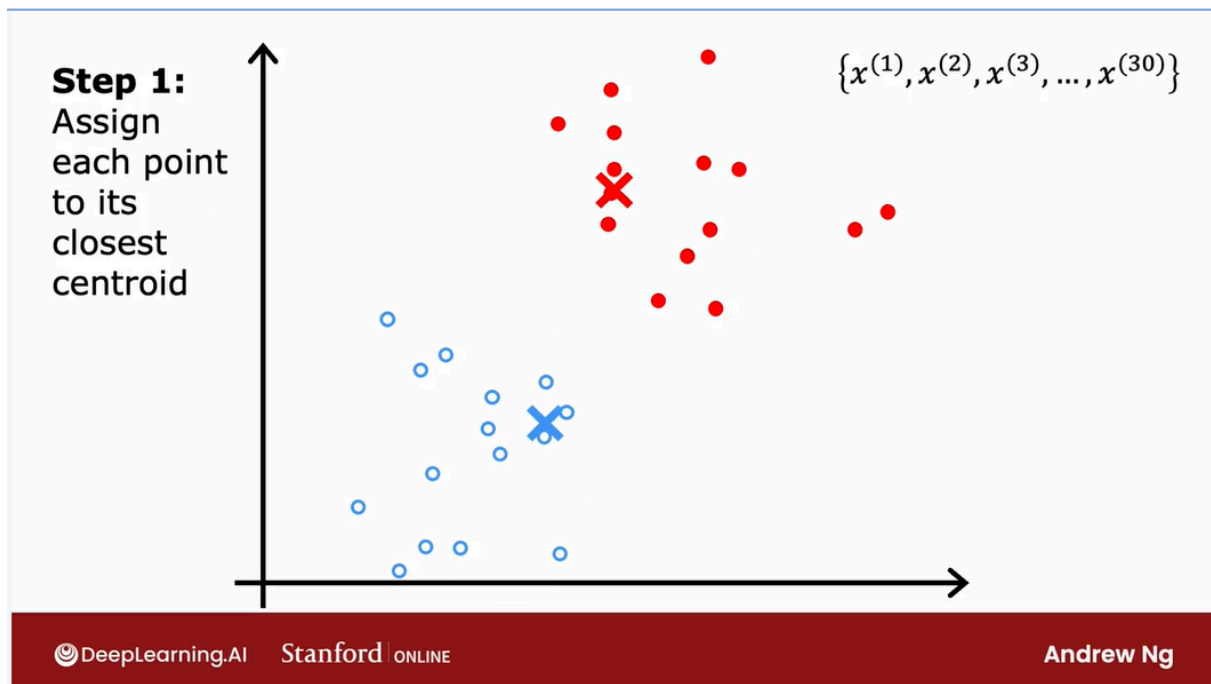


The second of the two steps that K-means does is, it'll look at all of the red points and [take an average of them](#). And it will move the red cross to whatever is the average location of the red dots, which turns out to be here. And so the red cross, that is the red cluster centroid will move here.

And then we do the same thing for all the blue dots. Look at all the blue dots, and take an average of them, and move the blue cross over there. So you now have a new location for the blue cluster centroid as well.

Then repeat the step 1 and 2:





If you keep on doing those two steps, you find that there are no more changes to the colors of the points or to the locations of the clusters centroids. And so this means that at this point the K-means clustering algorithm has converged.

Because applying those two steps over and over, results in no further changes to either the assignment to point to the centroids or the location of the cluster centroids.

K-means algorithm

K-means algorithm

Randomly initialize K cluster centroids

Repeat {

Assign points to cluster centroids

for $i = 1$ to m

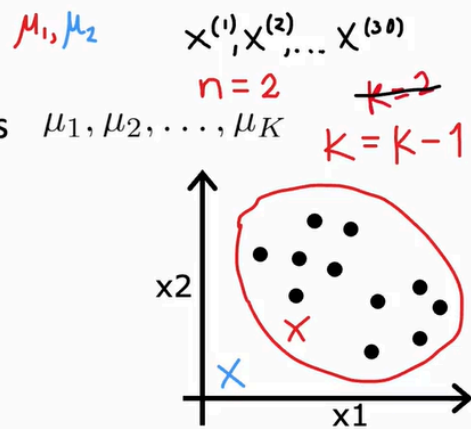
$c^{(i)} :=$ index (from 1 to K) of cluster centroid closest to $x^{(i)}$

Move cluster centroids

for $k = 1$ to K

$\mu_k :=$ average (mean) of points assigned to cluster k

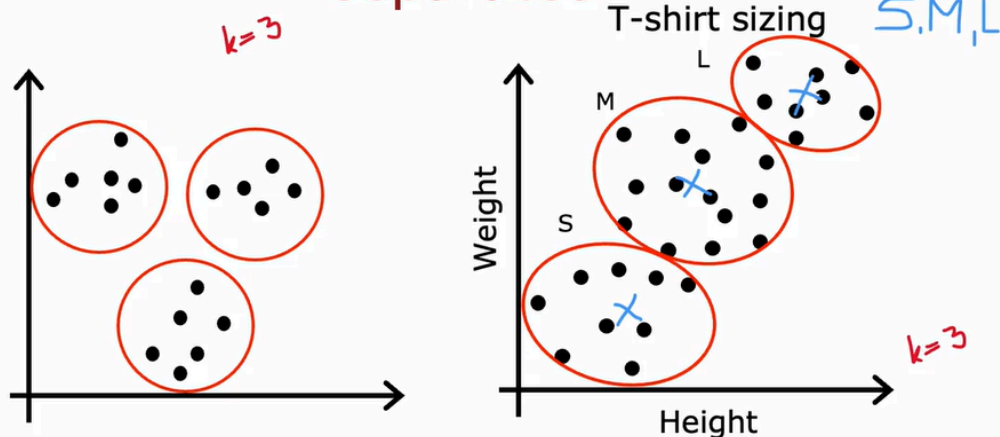
}



Now, there is one corner case of this algorithm, which is what happens if a cluster has zero training examples assigned to it.

In that case, the second step, μ_k , would be trying to compute the average of zero points. That's not well-defined. If that ever happens, the most common thing to do is to just eliminate that cluster. You end up with $K = K - 1$ clusters. Or if you really, really need K clusters an alternative would be to just randomly reinitialize that cluster centroid and hope that it gets assigned at least some points next time round. But it's actually more common when running K-means to just eliminate a cluster if no points are assigned to it.

K-means for clusters that are not well separated



Optimization objective

K-means optimization objective

$c^{(i)}$ = index of cluster $(1, 2, \dots, K)$ to which example $x^{(i)}$ is currently assigned

μ_k = cluster centroid k

$\mu_{c^{(i)}}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned

Cost function

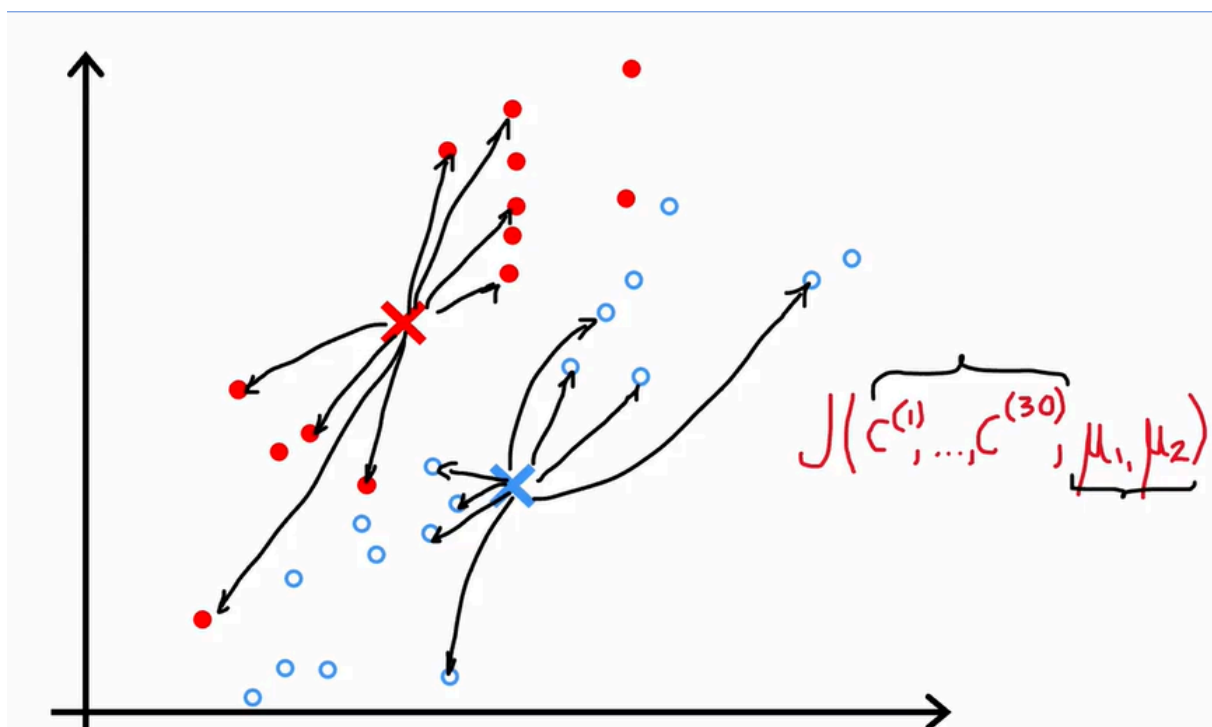
$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

$\min_{c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K} J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$

We want to minimize the cost function. The cost function of k-means is also called **Distortion function**.

- m : number of data points
- K : number of centroids

example:



Cost function for K-means

Cost function for K-means

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

Repeat {

Assign points to cluster centroids

for $i = 1$ to m

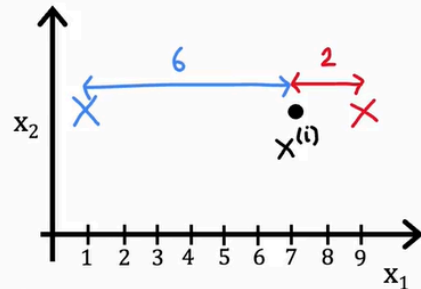
$c^{(i)} :=$ index of cluster
centroid closest to $x^{(i)}$

Move cluster centroids

for $k = 1$ to K

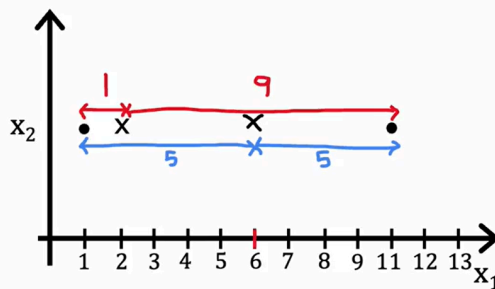
$\mu_k :=$ average of points in cluster k

}



viz:

Moving the centroid



$$\frac{1}{2}(1^2 + 11^2) = \frac{1}{2}(1 + 121) = 61$$

$$\frac{1}{2}(1 + 121) = 61$$

$$\frac{1}{2}(5^2 + 5^2) = 25$$

assume that:

point01 $\rightarrow x = 1$

point02 $\rightarrow x = 11$

distance from the centroid to p1 and p2 $\rightarrow 41$

take the average of p1 and p2 $\rightarrow$ new location of centroid is 6

→ the distance from the centroid to p1 and p2 is now 25 → smaller and optimizer

Initializing K-means

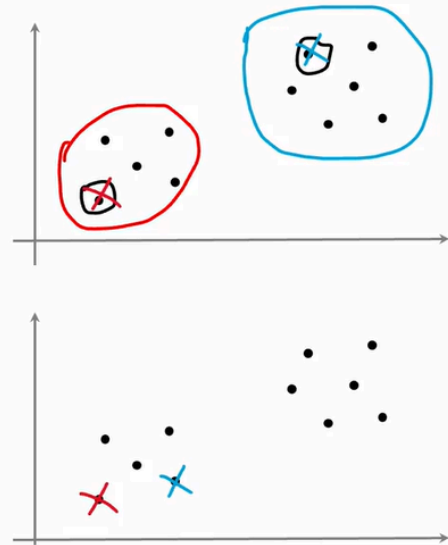
Random pick K training examples

Random initialization

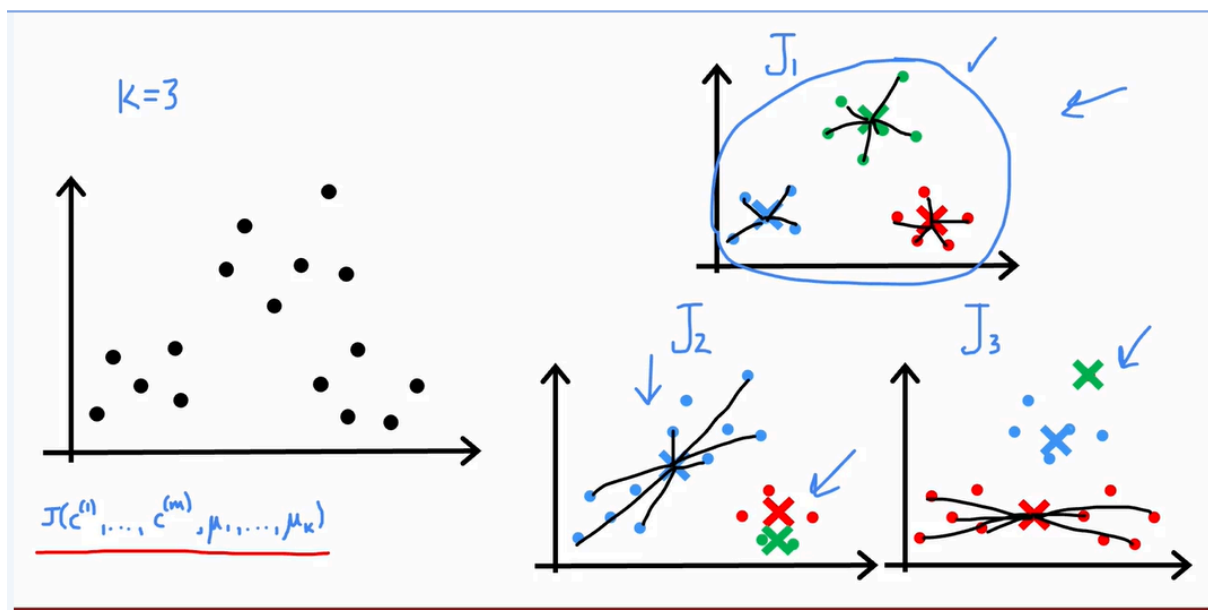
Choose $\underline{K} < \underline{m}$

Randomly pick K training examples.

Set $\mu_1, \mu_1, \dots, \mu_k$ equal to these K examples.



viz:



And here's another example of a local minimum J_3 , where a different random initialization course came in to find this clustering of the data into three

clusters, which again doesn't seem as good as the one that you saw up here on top.

So if you want to give k means multiple shots at finding the best local optimum. If you want to try multiple random initialization, so give it a better chance of finding this good clustering up on top. One other thing you could do with the K-means algorithm is to run it multiple times and then to try to find the best local optima.

And it turns out that if you were to run k means three times say, and end up with these three distinct clusterings. Then one way to choose between these three solutions, is to compute the cost function J for all three of these solutions, all three of these choices of clusters found by k means. And then to pick one of these three according to which one of them gives you the lowest value for the cost function J.

Random initialization

For $i = 1$ to 100 { 50-1000

Randomly initialize K-means. ← k random examples

Run K-means. Get $c^{(1)}, \dots, c^{(m)}, \mu_1, \mu_1, \dots, \mu_k$ ←

Computer cost function (distortion)

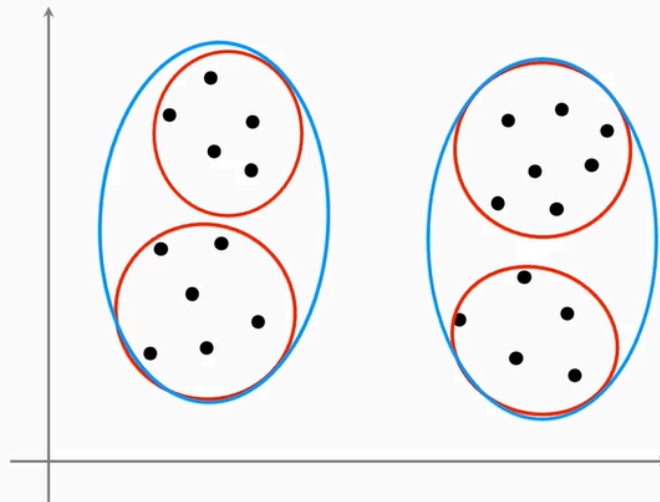
$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \mu_1, \dots, \mu_k)$ ←

}

Pick set of clusters that gave lowest cost J

Choosing the number of clusters (K)

What is the right value of K?



For a lot of clustering problems, the right value of K is truly ambiguous. If I were to show different people the same data set and ask, how many clusters do you see?

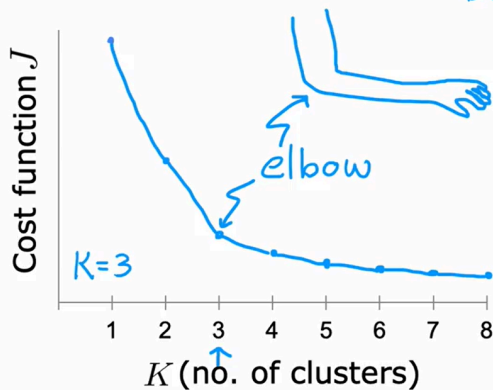
There will definitely be people that will say, it looks like there are two distinct clusters and they will be right. There would also be others that will see actually four distinct clusters. They would also be right. Because clustering is unsupervised learning algorithm you're not given the quote right answers in the form of specific labels to try to replicate. There are lots of applications where the data itself does not give a clear indicator for how many clusters there are in it.

Choosing the value of k

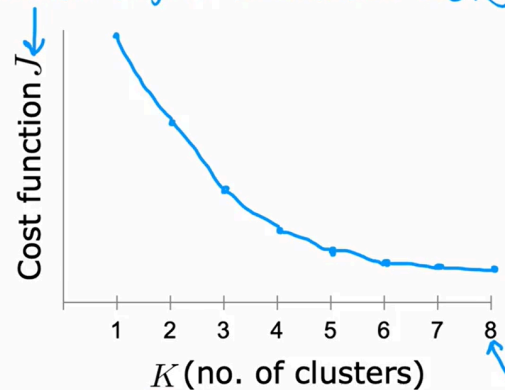
If you look at the academic literature on K-means, there are a few techniques to try to automatically choose the number of clusters to use for a certain application. I'll briefly mention one here that you may see others refer to, although I had to say, [I personally do not use this method myself](#).

Choosing the value of K

Elbow method



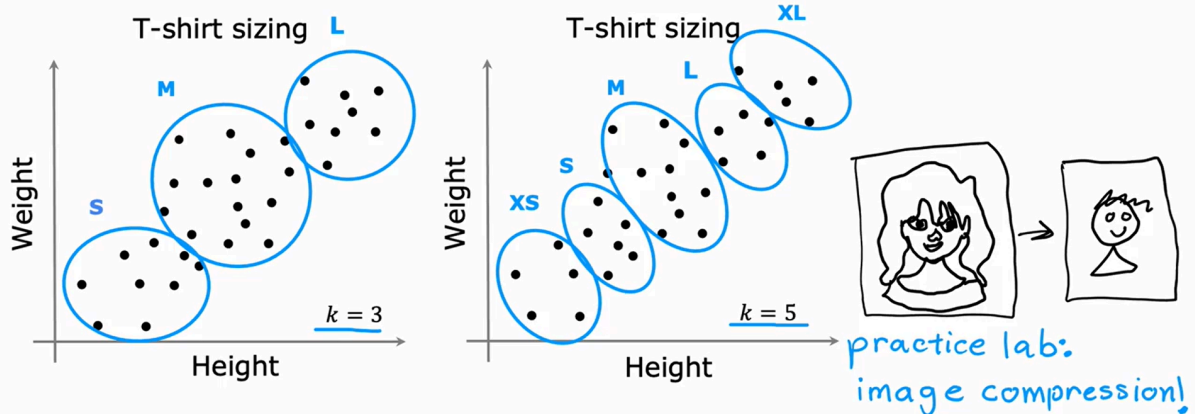
the right "K" is often ambiguous
Don't choose K just to minimize cost J



Plotting the cost function as a function of K could help, it could help you gain some insight. I personally hardly ever use the the elbow method myself to choose the right number of clusters because I think for a lot of applications, the right number of clusters is truly ambiguous.

Choosing the value of K

Often, you want to get clusters for some later (downstream) purpose. Evaluate K-means based on how well it performs on that later purpose.



One thing you could do is run K-means on this data set to find the clusters, in which case you may find clusters like that and this would be how you size your small, medium, and large t-shirts, but how many t-shirt sizes should there be? Well, it's ambiguous. If you were to also run K-means with five clusters, you might get clusters that look like this. This will let shoe size t-shirts according to extra small, small, medium, large, and extra large.

Both of these are completely valid and completely fine groupings of the data into clusters, but whether you want to use three clusters or five clusters can now be decided based on what makes sense for your t-shirt business.

Does a trade-off between how well the t-shirts will fit, depending on whether you have three sizes or five sizes, but there will be extra costs as well associated with manufacturing and shipping five types of t-shirts instead of three different types of t-shirts. What I would do in this case is to run K-means with K equals 3 and K equals 5 and then look at these two solutions to see based on the trade-off between fits of t-shirts with more sizes, results in better fit versus the extra cost of making more t-shirts where making fewer t-shirts is simpler and less expensive to try to decide what makes sense for the t-shirt business.

When you get to the programming exercise, you also see there an application of K-means to image compression. This is actually one of the most fun visual examples of K-means and there you see that there'll be a trade-off between the quality of the compressed image.

That is, how good the image looks versus how much you can compress the image to save the space. In that program exercise, you see that you can use that trade-off to maybe manually decide what's the best value of K based on how good do you want the image to look versus how large you want the compress image size to be.